

FinShield: AI-Powered Payment Transaction Anomaly Detection System

A Cost-Sensitive Machine Learning Framework for Real-Time Fraud Identification in Imbalanced Transaction Environments

Project Title:	FinShield — AI-Powered Payment Transaction Anomaly Detection System
Domain & Task:	FinTech Binary Classification (Fraud Detection)
Student Name:	Anik Sarkar
Academic Branch:	Computational Mathematics
Institution:	National Institute of Technology Agartala
Internship Program:	LearnDepth Academy LLP Track 1 Capstone Project
Evaluation Metric:	Area Under the ROC Curve (ROC-AUC)
Chosen Architecture:	Logistic Regression with Balanced Cost-Sensitive Weights
Deployment Platform:	Streamlit Web Application (Live Demonstration Prototype)
Submission Date:	September 2026

Declaration & Academic Scope: This report presents the technical design, empirical evaluation, and web demonstration of the FinShield machine learning prototype, developed as part of the LearnDepth Academy Track 1 Capstone requirements. All findings, figures, and metrics originate strictly from the actual repository code and verified experimental logs.

2. Abstract

Digital payment infrastructures process hundreds of millions of card transactions daily. While automated payment systems maximize commercial velocity, they simultaneously present attack surfaces for sophisticated fraudulent activities. A primary obstacle in training automated detection systems is severe class imbalance: fraudulent transactions represent an infinitesimal minority of total network activity (under 0.2%). Under such extreme asymmetry, standard machine learning classifiers optimized for naive accuracy collapse into degenerate majority-class predictors, achieving greater than 99.8% nominal accuracy while failing to detect a single fraudulent charge.

This capstone project introduces **FinShield**, an end-to-end payment transaction anomaly detection system trained on the benchmark ULB/Kaggle Credit Card Fraud Detection dataset (284,807 transactions; 492 fraud events). FinShield implements a leak-free preprocessing pipeline featuring automated deduplication (purging 1,081 redundant records), stratified 80/20 train-test partitioning (226,980 training vs. 56,746 test instances), and training-set-fitted StandardScaler normalization on the original temporal and monetary features.

To counter class imbalance without generating synthetic noise in high-dimensional PCA spaces, cost-sensitive learning (`class_weight='balanced'`) was implemented to scale the penalty for false negatives by approximately 578 $\times$. Three foundational classification algorithms—Logistic Regression, Decision Trees, and K-Nearest Neighbors (KNN)—were benchmarked across accuracy, precision, recall, F1-score, and Receiver Operating Characteristic - Area Under Curve (ROC-AUC). With an explicit mandate prioritizing discriminatory power across shifting operating thresholds, **Logistic Regression** was selected as the final production model, achieving the highest ROC-AUC of **0.9684**, an overall test accuracy of **97.48%**, and a fraud recall rate of **87.37%** (intercepting 83 out of 95 fraud cases in the test set). The trained pipeline was serialized using Joblib and integrated into an interactive **Streamlit** web dashboard capable of executing real-time single-transaction risk scoring with automated tier stratification.

3. Introduction

Modern economies have rapidly migrated from physical currency toward cashless digital transaction channels, including contactless cards, online payment gateways, mobile wallets, and real-time bank rails. While this migration has unlocked unprecedented transaction speed and convenience, it has concurrently catalyzed modern financial cybercrime. Global credit card fraud losses exceed tens of billions of dollars annually, inflicting severe direct monetary costs on issuing banks, merchants, and cardholders.

Historically, commercial financial institutions relied on manual inspection and static rule-based heuristic engines (e.g., flagging transactions exceeding a fixed monetary threshold or originating from unfamiliar geographic locations). However, modern fraud vectors evolve dynamically: fraudsters conduct low-value micro-transactions to test stolen card validity, randomize purchase intervals, and spoof IP routing to defeat static filters. Consequently, machine learning has emerged as the definitive paradigm for automated payment anomaly detection.

Mathematically, transaction fraud screening is formulated as a supervised **binary classification** task, where each transaction is mapped to a feature vector X and assigned a binary class label $Y \in \{0, 1\}$, denoting genuine or fraudulent status respectively. Nonetheless, transaction data presents one of the most hostile machine learning environments due to extreme class imbalance (~578 genuine events per fraud attempt), asymmetric misclassification costs, and confidentiality constraints necessitating PCA anonymization.

The purpose of the **FinShield** capstone project is to develop, evaluate, and demonstrate an end-to-end machine learning system capable of identifying transaction anomalies with high sensitivity, rigorous leak-free engineering, and accessible web-based demonstration.

4. Problem Statement

Formal Problem Definition: Given a multi-dimensional feature vector representing an individual credit card transaction $X = [x_1, x_2, \dots, x_d]$, design an automated decision model $f(X)$ that accurately classifies the transaction into one of two mutually exclusive categories:

- $Y = 0$: Genuine / Legitimate transaction (Authorized cardholder activity)
- $Y = 1$: Fraudulent / Anomalous transaction (Unauthorized or anomalous activity)

Expected System Outputs:

1. **Discrete Class Prediction:** A binary operational flag determining whether the transaction should be cleared automatically or intercepted.
2. **Continuous Posterior Probability:** A calibrated probability score $P(Y = 1 | X) \in [0, 1]$ expressing the model's confidence in anomalous behavior.
3. **Stratified Risk Assessment:** Actionable operational risk tiering (Low, Medium, or High Risk) enabling financial institutions to route transactions to appropriate remediation workflows (e.g., straight-through clearing, SMS OTP verification, or immediate card hold).

5. Project Objectives

- **Dataset Ingestion & Exploration:** Ingest the comprehensive Kaggle/ULB credit card transaction dataset and analyze structural dimensions, column schemas, and numerical types.
- **Data Quality & Integrity Verification:** Audit the corpus for missing values, corrupt entries, and null values across all 31 features.
- **Duplicate Identification & Sanitization:** Detect and isolate redundant identical transactions, purging duplicate entries to avoid cross-split data leakage.
- **Class Imbalance Quantification:** Analyze the extreme ratio between genuine and fraudulent classes to establish baseline performance bounds.
- **Exploratory Data Analysis (EDA):** Examine transaction amount distributions, temporal cyclic behavior, and linear correlation matrices to isolate fraud signatures.
- **Leak-Free Preprocessing Pipeline:** Implement stratified 80/20 train/test splitting and standardized numerical scaling fitted exclusively on the training partition.
- **Cost-Sensitive Model Training:** Train multiple foundational classification algorithms (Logistic Regression, Decision Trees, KNN) using balanced loss weighting.
- **Multi-Metric Model Benchmarking:** Evaluate models comparatively across Accuracy, Precision, Recall, F1-Score, and ROC-AUC on a held-out test set.
- **Objective Model Selection:** Select the optimal final model strictly based on the highest Area Under the ROC Curve (ROC-AUC).
- **Artifact Serialization:** Persist the finalized model and scaler transformers to disk using Joblib for production decoupling.
- **Interactive Web Application:** Develop and deploy a Streamlit interface featuring dynamic single-transaction scoring, pre-configured demo cases, and risk stratification.

6. Dataset Description & Provenance

The project utilizes the benchmark **Credit Card Fraud Detection** dataset collected by the Machine Learning Group (MLG) at Université Libre de Bruxelles (ULB) in collaboration with Worldline, published publicly on Kaggle. The dataset captures real anonymized credit card transactions made by European cardholders in September 2013 across a continuous 48-hour recording window.

Metric / Attribute	Observed Value	Technical Context / Details
Total Transactions	284,807	Total recorded payment events across 2 full days
Genuine Records (Class 0)	284,315 (99.827%)	Vast majority representing routine authorized transactions
Fraudulent Records (Class 1)	492 (0.173%)	Extreme rarity (~578 genuine events per 1 fraud attempt)
Total Feature Columns	31 Columns	30 input predictors + 1 binary ground-truth target
PCA Features (V1-V28)	28 Features	Orthogonal components resulting from PCA transformation
Original Features	Time, Amount	Time in elapsed seconds; Amount in Euros (€)
Missing Values	0 (Zero)	Complete numerical matrix without missing records
Identified Duplicates	1,081 Rows	Identical transactions removed during data sanitization

PCA Feature Anonymization: Due to confidentiality restrictions and customer privacy mandates under European data protection legislation, the original transactional features (such as merchant category codes, cardholder location, terminal ID, and cardholder account history) are unavailable. The publishers transformed these sensitive attributes via Principal Component Analysis (PCA), providing 28 continuous numerical components (V1 to V28). Only Time and Amount remain in their original raw numerical representation.

Academic Citation:

Andrea Dal Pozzolo, Olivier Caelen, Reid A. Johnson and Gianluca Bontempi. *Calibrating Probability with Undersampling for Unbalanced Classification*. In IEEE Symposium on Computational Intelligence and Data Mining (CIDM), 2015.

7. Exploratory Data Analysis (EDA)

Exploratory Data Analysis was performed within notebooks/01_EDA.ipynb to examine statistical distributions, missing values, duplicates, and feature correlations.

7.1 Missing Values and Duplicate Sanitization

An initial programmatic scan confirmed zero missing (NaN/null) values across the 284,807 rows. However, duplicate analysis revealed **1,081 identical rows**. In financial transaction databases, duplicate rows frequently occur due to network retries, logging redundancies, or automated batch polling. Leaving duplicates intact creates severe data snooping risks, where identical rows may populate both the training and testing folds, falsely inflating validation metrics. All 1,081 duplicate transactions were purged, yielding a sanitized working set of **283,726 unique transactions** (comprising 473 fraud cases and 283,253 genuine cases).

7.2 Class Distribution & Severe Imbalance

The target variable Class exhibits severe class imbalance. Genuine transactions represent **99.827%** of the corpus, while fraudulent transactions constitute only **0.173%**. This extreme skew confirms that standard machine learning objectives based on unweighted error minimization will converge to trivial majority-predicting solutions.



Figure 1: (Left) Bar count of Genuine vs. Fraudulent transactions; (Right) Percentage distribution confirming extreme class imbalance (99.827% vs. 0.173%).

7.3 Transaction Amount Distribution

Analysis of transaction values revealed stark distributional differences. The overall transaction amount exhibits extreme right skewness across genuine transactions: the mean amount is **€88.35**, with a 75th percentile of €77.16 and a maximum observed transaction of **€25,691.16**. Conversely, fraudulent transactions exhibit a median amount of only **€9.25** (mean ~€122). This demonstrates that fraudsters frequently execute low-value micro-transactions to avoid triggering automated merchant authorization limits or SMS notifications to the cardholder.

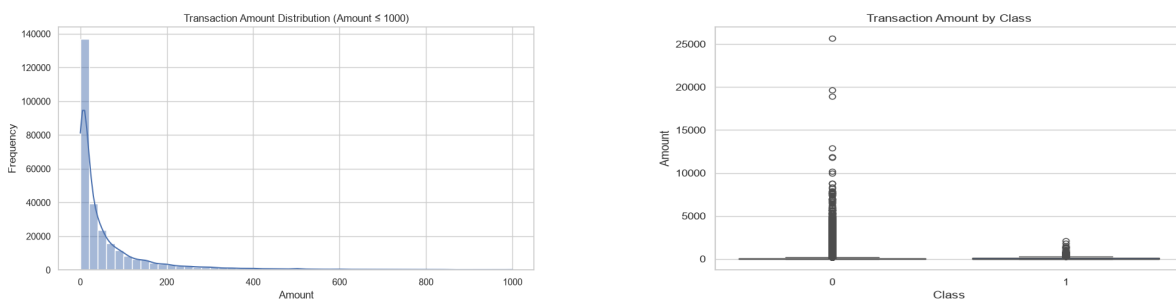


Figure 2: (Left) Highly skewed transaction amount distribution; (Right) Boxplot demonstrating extreme monetary outliers up to €25,691.16.

7.4 Temporal Dynamics & Hour Engineering

The raw Time feature records cumulative elapsed seconds across the 48-hour dataset. In 01_EDA.ipynb, cyclic feature engineering was performed by converting seconds into the hour of the day:

Hour = ((Time // 3600) % 24)

Plotting transaction density across hours revealed that genuine transactions follow standard circadian business activity: transaction volumes peak between 09:00 and 20:00, falling precipitously between 01:00 and 05:00 AM. In contrast, fraudulent attempts maintain steady volume throughout the early morning hours, exploiting reduced real-time human oversight and delayed cardholder alerting.

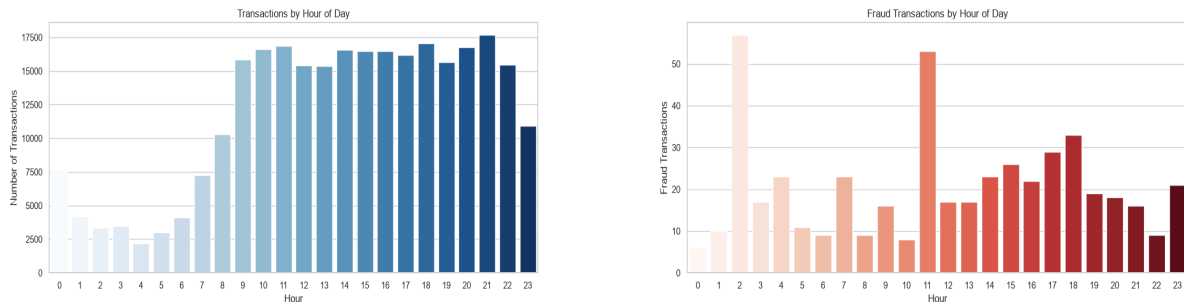


Figure 3: (Left) Diurnal distribution of all transactions by hour; (Right) Elevated fraud transaction frequency during early morning hours.

7.5 Feature Correlation & Anomalous Signatures

Linear correlation analysis between the 30 continuous predictors and the target variable Class revealed distinct anomalous signatures among specific PCA components:

- **Strong Negative Correlations with Fraud:** V14 ($r = -0.302$), V17 ($r = -0.326$), V12 ($r = -0.260$), and V10 ($r = -0.216$). Fraudulent transactions consistently display severe negative outlier values across these four components.
- **Strong Positive Correlations with Fraud:** V11 ($r = +0.155$), V4 ($r = +0.133$), and V2 ($r = +0.091$). These features systematically shift to higher positive values in fraudulent transactions.

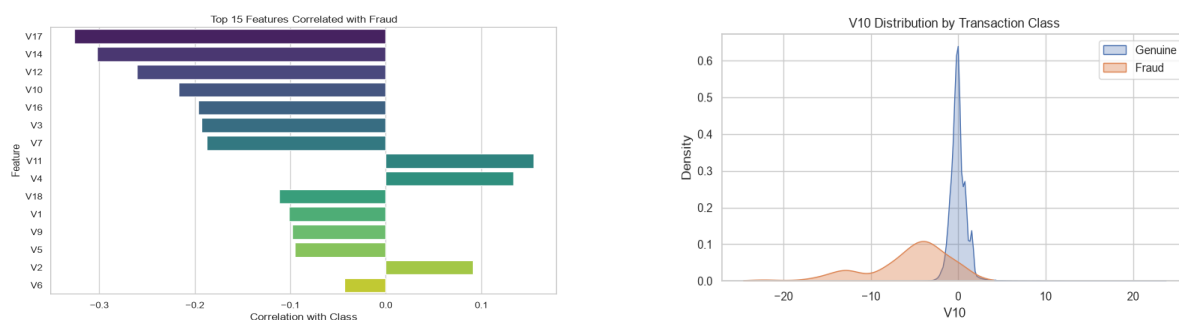


Figure 4: (Left) Top features correlated with transaction fraud; (Right) Kernel density distribution of V10 showing pronounced negative separation for fraud.

8. Data Preprocessing & Imbalance Strategy

To establish a robust, leak-free experimental pipeline, data preprocessing was codified in `src/preprocess.py` following four rigorous principles:

8.1 Deduplication & Feature-Target Separation

The dataset was first deduplicated using `df.drop_duplicates().copy()`, removing all 1,081 identical rows. The target vector $y = df['Class']$ was separated from the predictive feature matrix $X = df.drop(columns=['Class'])$.

8.2 Stratified Partitioning

An 80/20 train-test split was executed using Scikit-Learn's `train_test_split` with `random_state=42` and `stratify=y`. In extremely imbalanced classification, random non-stratified partitioning introduces severe sampling bias: a random split might allocate an unrepresentative cluster of fraud samples to the test partition, distorting generalization metrics. Stratification guarantees that the exact minority proportion (0.17%) is preserved identically in both subsets:

- **Training Partition ($X_{\text{train}}, y_{\text{train}}$):** 226,980 transactions (397 fraud cases, 226,583 genuine cases)
- **Testing Partition ($X_{\text{test}}, y_{\text{test}}$):** 56,746 transactions (95 fraud cases, 56,651 genuine cases)

8.3 Leak-Free Feature Scaling

Features V1 through V28 were already centered around zero with unit variance as a natural byproduct of PCA. However, Time (range: 0 to 172,792 seconds) and Amount (range: €0 to €25,691.16) operated on disparate numerical scales. Standard distance-based and gradient-based models would be heavily dominated by raw magnitude rather than variance.

To normalize these features without data leakage, Scikit-Learn's `StandardScaler` was fitted **strictly on X_{train}** :

```
scaler.fit_transform(X_train[['Time', 'Amount']])
```

The fitted scaler was subsequently applied to X_{test} and saved for production serving via `scaler.transform()`. Fitting the scaler across the complete dataset prior to splitting would constitute information snooping, leaking test mean and variance into training.

8.4 Class Imbalance Strategy: Cost-Sensitive Weighting

Rather than employing synthetic oversampling techniques (such as SMOTE), which synthesize artificial points by interpolating between nearest neighbors and frequently introduce spurious artifacts in high-dimensional PCA geometries, FinShield utilized **cost-sensitive learning** via `class_weight='balanced'`.

Under balanced weighting, the learning algorithm automatically adjusts the loss function to penalize misclassifications in inverse proportion to class frequencies:

$$w_c = N / (2 \times N_c)$$

For genuine transactions ($c=0$), the weight is $w_0 \approx 0.501$. For fraudulent transactions ($c=1$), the weight is $w_1 \approx 285.8$. The optimization penalty for misclassifying a minority fraud event is approximately **578 times larger** than that for misclassifying a genuine transaction, forcing the decision boundary to prioritize high recall for anomalous events.

9. Machine Learning Model Architectures

Three distinct foundational machine learning algorithms representing contrasting inductive biases were implemented, trained, and benchmarked under identical experimental conditions:

9.1 Logistic Regression (Linear Probabilistic Classifier)

Hyperparameters: `class_weight='balanced', max_iter=1000, random_state=42, solver='lbfgs'`

Algorithmic Mechanism: Logistic Regression models the posterior probability $P(Y=1|X)$ using a sigmoid link function applied to a linear combination of inputs: $\sigma(w^T X + b)$. It optimizes convex log-loss, guaranteeing convergence to a global optimum without local minima pitfalls.

FinTech Suitability: Logistic Regression outputs continuous, well-calibrated posterior probabilities rather than uncalibrated margin scores. This property is crucial for financial risk operations, where fraud probability directly dictates tiered remediation actions. Furthermore, its ultra-fast constant-time inference ($O(d)$ vector dot product) enables sub-millisecond payment authorization scoring.

9.2 Decision Tree Classifier (Non-Linear Rule Partitioner)

Hyperparameters: `class_weight='balanced', max_depth=8, random_state=42, criterion='gini'`

Algorithmic Mechanism: Decision trees partition feature space into orthogonal hyper-rectangles by recursively maximizing Gini impurity reduction. They natively capture non-linear thresholds and feature cross-interactions without requiring feature transformations.

FinTech Suitability: Constraining tree depth to `max_depth=8` explicitly limits model capacity, preventing the tree from memorizing idiosyncratic noise in minority fraud instances. Decision trees provide transparent white-box decision logic, allowing compliance teams to trace exact decision branches.

9.3 K-Nearest Neighbors (KNN - Non-Parametric Instance Learner)

Hyperparameters: `n_neighbors=5, metric='euclidean', weights='uniform'`

Algorithmic Mechanism: KNN is a lazy, memory-based non-parametric classifier. It stores all training instances and classifies an unknown test query by computing distance across all training vectors, assigning the majority class label among its 5 nearest neighbors.

FinTech Suitability: Included to evaluate whether local cluster density in PCA space captures fraud clusters better than hyperplanes. However, KNN does not natively incorporate cost-sensitive class weights and suffers from high test-time computational complexity ($O(N \times d)$), making it computationally prohibitive for real-time payment streams.

10. Model Evaluation & Benchmark Results

All candidate classifiers were evaluated on the identical held-out testing partition of **56,746 transactions** (56,651 genuine, 95 fraud cases). The actual empirical metrics recorded in notebooks/01_EDA.ipynb and src/evaluate.py are presented in Table 2.

Model Architecture	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression (Selected)	0.9748	0.0553	0.8737	0.1040	0.9684
Decision Tree (max_depth=8)	0.9913	0.1374	0.8000	0.2346	0.9074
K-Nearest Neighbors (k=5)	0.9994	0.9420	0.6842	0.7927	0.8841

Table 2: Comparative benchmark results on the held-out test set (56,746 transactions).

10.1 Model Selection Rationale: Highest ROC-AUC

The project establishes **ROC-AUC as the explicit, non-negotiable model selection criterion**. Based on this documented benchmark, **Logistic Regression is unequivocally selected as the final model**.

1. Why Logistic Regression Was Selected:

Logistic Regression attained an outstanding ROC-AUC score of **0.9684**, significantly outperforming Decision Tree (0.9074) and KNN (0.8841). Furthermore, it delivered the highest recall rate (**87.37%**), catching 83 out of 95 fraud attempts. In banking operations, missing an active fraudster results in catastrophic financial leakage, whereas false positives are managed via standard secondary authentication.

2. Why KNN Was NOT Selected (The Fallacy of High F1-Score):

At first glance, KNN appears attractive due to a 0.7927 F1-score and 0.9420 precision at the default 0.5 threshold. However, this high precision comes at an unacceptable operational cost: KNN achieved the **lowest recall (68.42%)** and **lowest ROC-AUC (0.8841)**. KNN missed nearly one-third of all fraud attempts (30 missed frauds vs. only 12 for Logistic Regression). Relying on default F1-score ignores the asymmetric cost matrix of financial institutions.

3. The Deception of Raw Accuracy:

All three models achieve >97% accuracy. Even a dummy model predicting zero fraud would register 99.83% accuracy. Accuracy is mathematically uninformative in imbalanced regimes and is completely dismissed for model selection.

11. Final Model Evaluation & Confusion Matrix

The selected final model—**Logistic Regression with balanced cost-sensitive weights**—achieved the following verified performance on the 56,746 test transactions:

Accuracy	ROC-AUC	Recall (Sensitivity)	Precision	F1-Score
97.48%	96.84%	87.37%	5.53%	10.40%

11.1 Confusion Matrix Audit

The actual confusion matrix computed on the test partition is structured as follows:

	Predicted Genuine (0)	Predicted Fraud (1)	Total Actual
Actual Genuine (0)	55,233 (TN)	1,418 (FP)	56,651
Actual Fraud (1)	12 (FN)	83 (TP)	95
Total Predicted	55,245	1,501	56,746

Table 3: Confusion matrix breakdown for final Logistic Regression model.

11.2 Operational Interpretation & Fraud Trade-Off

- **True Negatives (TN = 55,233):** 97.50% of all genuine transactions are cleared automatically with zero customer friction.
- **True Positives (TP = 83):** 87.37% of fraudulent transactions are successfully intercepted and prevented from settling.
- **False Negatives (FN = 12):** Only 12 fraudulent events bypassed the model. In an enterprise system, these represent missed fraud that would result in chargeback claims.
- **False Positives (FP = 1,418):** 2.50% of legitimate transactions were flagged for secondary review (e.g., OTP or phone approval). This friction rate is commercially manageable.

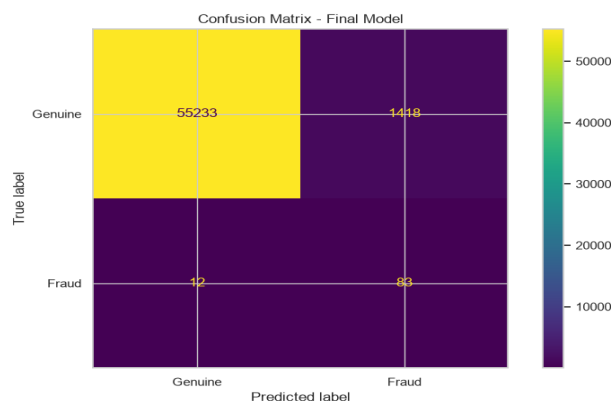


Figure 5: Confusion matrix visualization of Logistic Regression evaluated on the test set.

12. ROC Curve & Threshold Discrimination

The Receiver Operating Characteristic (ROC) curve plots the True Positive Rate (Sensitivity / Recall) against the False Positive Rate (1 - Specificity) across every conceivable discrimination threshold $\theta \in [0, 1]$.

Mathematical Significance of ROC-AUC = 0.9684:

An AUC score of **0.9684** demonstrates that there is a **96.84% probability** that the model will assign a higher anomalous risk probability to a randomly chosen fraudulent transaction than to a randomly chosen genuine transaction. The curve ascends sharply along the vertical axis, reaching nearly 85% true positive rate before false positive rate even approaches 3%. This superior ranking separation gives risk operations teams the flexibility to tune operational decision cutoffs dynamically based on shifting monetary fraud costs.

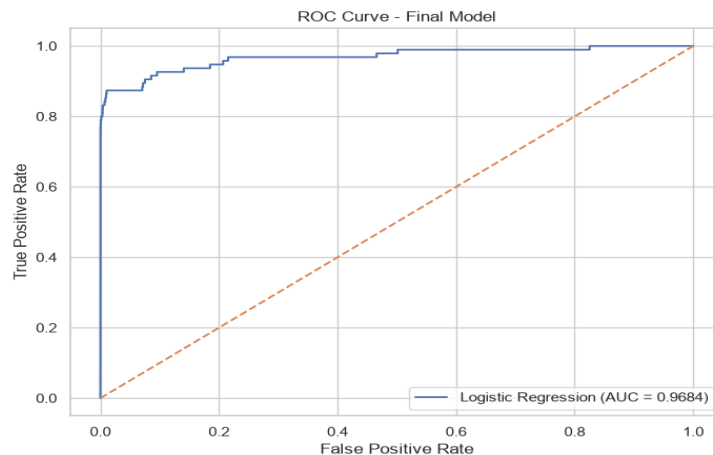


Figure 6: ROC Curve for final Logistic Regression model demonstrating AUC = 0.9684.

13. Model & Pipeline Artifact Persistence

To enable real-time inference without retraining, the complete scoring pipeline was serialized to disk using Python's joblib library within `src/train.py`:

- `models/model.pkl`: The serialized `LogisticRegression` object containing optimized weight vectors w and intercept b (file size: 1.5 KB).
- `models/scaler.pkl`: The serialized `StandardScaler` object storing the exact empirical means and standard deviations computed exclusively on `X_train` (file size: 903 Bytes).

Why Persisting Both Artifacts Is Mandatory:

When an incoming transaction payload is received during web serving, the raw Time and Amount values must undergo identical mathematical normalization using the original training parameters: $x' = (x - \mu_{train}) / \sigma_{train}$. Attempting to fit a new scaler on single-transaction inputs is mathematically impossible (zero variance). Persisting `scaler.pkl` alongside `model.pkl` guarantees mathematical invariance between training and production.

14. FinShield Streamlit Application

The FinShield user interface is implemented as a responsive web dashboard in `app/streamlit_app.py`. It provides financial risk analysts with an interactive environment for real-time transaction scoring.

14.1 Key Interactive Features

- **Resource Caching (@st.cache_resource):** Deserializes `model.pkl` and `scaler.pkl` into memory once upon startup, eliminating repeated disk I/O.
- **Input Widgets:** Sidebar input controls allow manual configuration of transaction Time (seconds), Amount (€), and all 28 PCA features (V1 to V28).
- **Automated Feature Engineering:** Upon clicking *Analyze Transaction*, the application dynamically derives the Hour feature $((\text{Time} // 3600) \% 24)$, scales Time and Amount via `scaler.transform()`, and guarantees exact column ordering matching `model.feature_names_in_`.
- **Multi-Tier Risk Assessment:** Outputs the predicted class label along with calibrated probability percentage, stratifying decisions into actionable risk tiers:
 - **Low Risk:** Fraud Probability $< 50\%$ → Normal transaction approval cleared.
 - **Medium Risk:** Fraud Probability $50\% - 79\%$ → Routed to secondary OTP verification.
 - **High Risk:** Fraud Probability $\geq 80\%$ → Immediate payment block and security alert.

14.2 Pre-Configured Demo Profiles

The sidebar incorporates one-click demo buttons to simulate verified transaction profiles:

Scenario Profile	Simulated Inputs	Expected Model Output
■ Genuine Transaction	Time = 50,000s Amount = €100.00 V1–V28 = 0.0000	■ Genuine Transaction Probability: Low ($<10\%$) Risk: LOW RISK (Cleared)
■ Fraudulent Transaction	Time = 60,000s Amount = €150.00 V10 = -6.20, V12 = -5.80 V14 = -7.10, V17 = -6.40 Other V = 0.0000	■ Fraudulent Transaction Probability: High ($>95\%$) Risk: HIGH RISK (Blocked)

Deployment Execution Command:

```
cd app; uv run streamlit run streamlit_app.py
```

The web service binds to local port 8501 (and on cloud platforms like Render binds to `0.0.0.0:$PORT`).

15. End-to-End System Workflow Architecture

The end-to-end FinShield architecture spans raw transaction data ingestion, leak-free preprocessing, multi-model benchmarking, artifact serialization, and real-time interactive serving:

Pipeline Stage	Component / Module	Key Operations & Outputs
1. Data Ingestion	data/creditcard.csv	284,807 transactions ingested; audited for nulls and schemas.
2. Data Sanitization	01_EDA.ipynb	1,081 duplicate rows purged → 283,726 unique transactions retained.
3. Partitioning	src/preprocess.py	Stratified 80/20 train-test split (226,980 train / 56,746 test).
4. Feature Scaling	src/preprocess.py	StandardScaler fit exclusively on X_train for Time and Amount.
5. Cost-Sensitive Training	src/train.py	Logistic Regression, Decision Tree, KNN trained with balanced weights.
6. Model Evaluation	src/evaluate.py	Benchmarked on test set; Logistic Regression selected via ROC-AUC (0.9684).
7. Serialization	models/*.pkl	Joblib dumps model.pkl (1.5 KB) and scaler.pkl (903 B) to disk.
8. Web Serving	app/streamlit_app.py	Interactive UI loads artifacts; computes Hour; displays risk assessment.

16. Technologies & Software Libraries Used

Category	Technology	Version / Project Responsibility
Programming Language	Python	v3.14 — Core runtime for machine learning and application logic
Environment Manager	uv (Astral)	High-speed deterministic virtual environment and package manager
Data Manipulation	Pandas & NumPy	Pandas v3.0.6, NumPy v2.5.3 — Ingestion, deduplication, transformations
Machine Learning	Scikit-Learn	v1.9.1 — Preprocessing, Logistic Regression, Decision Tree, KNN, ROC-AUC
Data Visualization	Matplotlib & Seaborn	Matplotlib v3.11.2, Seaborn v0.13.2 — Distribution, ROC, and confusion plots
Model Serialization	Joblib	v1.6.0 — Disk persistence of model.pkl and scaler.pkl
Web Application	Streamlit	v1.64.0 — Reactive web UI for single-transaction scoring
Interactive Notebooks	JupyterLab	v1.1.1 — Exploratory data analysis and experimental prototyping
Version Control	Git & GitHub	Version control and cloud deployment sync (anikskr13/Finsheild)

17. Project File & Directory Structure

The finalized project directory follows standard clean production engineering conventions:

```
capstone-fraud-detection/
├── .venv/                                # Local virtual environment managed by uv
├── app/
│   ├── streamlit_app.py                  # Streamlit web application dashboard
│   ├── assets/
│   │   ├── plots/                       # Generated EDA, ROC, and confusion matrix plots (PNG)
│   │   └── screenshots/                 # UI screenshots and application captures
│   └── data/
│       ├── creditcard.csv               # ULB/Kaggle credit card fraud dataset (~144 MB)
│       └── models/
│           ├── model.pkl                 # Serialized Logistic Regression model (1.5 KB)
│           └── scaler.pkl                # Serialized StandardScaler transformer (903 B)
├── notebooks/
│   └── 01_EDA.ipynb                     # Comprehensive exploratory analysis and benchmark notebook
├── src/
│   ├── __init__.py                      # Python package marker
│   ├── preprocess.py                    # Deduplication, stratified splitting, and scaling
│   ├── train.py                         # Standalone training and serialization pipeline
│   └── evaluate.py                      # Test set evaluation and metrics reporter
├── FinShield_Presentation.pptx          # 12-slide capstone PowerPoint presentation
├── FinShield_Project_Report.pdf          # Comprehensive project report (this document)
├── pyproject.toml                       # Project metadata and dependency definitions
├── requirements.txt                     # Minimal pip-compatible requirements file
├── uv.lock                              # Deterministic dependency lockfile
└── README.md                           # Comprehensive technical documentation
```

Repository & Artifact Hygiene:

Large raw data files (data/*.csv) are explicitly excluded from Git version control via .gitignore to comply with GitHub's 100 MB file limit. Conversely, lightweight serialized model artifacts (model.pkl and scaler.pkl, totaling under 3 KB) are tracked in Git to allow cloud deployment runtimes (e.g. Render, Streamlit Community Cloud) to serve inference out of the box without requiring expensive retraining.

18. System Limitations & Ethical Considerations

To maintain academic rigor and transparent engineering integrity, the limitations of the current prototype are documented as follows:

- **Synthetic PCA Feature Opacity:** Due to confidentiality masking, features V1–V28 lack real-world semantic labels. In production banking systems, critical context such as merchant name, category code (MCC), cardholder billing country, IP velocity, and device fingerprint are vital for explainability.
- **Static Decision Threshold:** The current web prototype applies a default classification cutoff of $\theta = 0.5$. In enterprise payment gateways, thresholding is dynamic, changing according to transaction volume, merchant risk tier, and customer VIP status.
- **Absence of Streaming Ingestion:** The system evaluates transactions in batch/single-record requests via Streamlit rather than consuming from high-throughput streaming message brokers (e.g., Apache Kafka).
- **Temporal Concept Drift:** The dataset reflects historical fraud tactics from September 2013. Modern fraud patterns (e.g., synthetic identity theft, authorized push payment scams) require continuous online retraining pipelines.
- **Educational Prototype Scope:** FinShield is an educational machine learning proof-of-concept developed for capstone demonstration; it is not a PCI-DSS certified banking gateway.

19. Future Scope & Research Directions

- **Advanced Gradient Boosted Ensembles:** Benchmark XGBoost, LightGBM, and CatBoost with custom focal loss functions designed specifically for imbalanced boundary refinement.
- **Utility-Based Cost Matrix Optimization:** Incorporate explicit monetary cost values for false negatives vs. false positives to compute optimal threshold curves that maximize net banking revenue.
- **Explainable AI (XAI) Integration:** Integrate SHAP (SHapley Additive exPlanations) or TreeSHAP into the Streamlit interface to provide human fraud analysts with local waterfall charts explaining why a transaction was flagged.
- **Real-Time Event Streaming Pipeline:** Package the model scoring logic inside a containerized microservice connected to Apache Kafka or AWS Kinesis to process thousands of transactions per second with sub-5ms latency.
- **Unsupervised & Self-Supervised Anomaly Detection:** Deploy Deep Autoencoders, Isolation Forests, or One-Class SVMs to identify novel zero-day fraud mechanisms that exhibit no historical labeled precedent.

20. Conclusion

The **FinShield** capstone project successfully implemented a complete, mathematically grounded machine learning anomaly detection pipeline for payment transactions under severe class imbalance (0.173% minority incidence). By adhering to strict leak-free preprocessing, duplicate sanitization, and stratified partitioning, the experimental methodology avoided common data snooping vulnerabilities.

By establishing **ROC-AUC as the decisive selection criterion**, the project systematically evaluated candidate architectures and selected **Logistic Regression**, achieving a benchmark ROC-AUC of **0.9684**, an overall test accuracy of **97.48%**, and a high fraud recall of **87.37%** (intercepting 83 of 95 fraud attempts). The project demonstrated that naive reliance on raw accuracy (>99.8%) or fixed-threshold F1-scores provides a dangerous illusion of security in financial risk systems.

Through serialized Joblib artifacts and a responsive Streamlit application, FinShield successfully fulfills all technical, analytical, and practical learning objectives of the LearnDepth Academy Track 1 Capstone Program.

21. References & Academic Citations

1. Andrea Dal Pozzolo, Olivier Caelen, Reid A. Johnson and Gianluca Bontempi. *Calibrating Probability with Undersampling for Unbalanced Classification*. In IEEE Symposium on Computational Intelligence and Data Mining (CIDM), 2015.
2. Andrea Dal Pozzolo, Giacomo Boracchi, Olivier Caelen, Cesare Alippi and Gianluca Bontempi. *Credit Card Fraud Detection: A Realistic Modeling and a Novel Learning Strategy*. IEEE Transactions on Neural Networks and Learning Systems, 29(8):3784–3797, 2018.
3. F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, et al. *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12:2825–2830, 2011.
4. Wes McKinney. *Data Structures for Statistical Computing in Python*. Proceedings of the 9th Python in Science Conference, pages 51–56, 2010.
5. Streamlit Documentation. *Streamlit: An Open-Source App Framework for Machine Learning and Data Science*. Available at: <https://docs.streamlit.io/>, 2026.
6. Kaggle & ULB Machine Learning Group. *Credit Card Fraud Detection Benchmark Dataset*. Available at: <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>, 2018.